A Semantic Entropy Approach for Reliable Policy Question Answering

Pranjal Shinde, Saanvi Pathak, Tanishka Kumar, and Aditi Bhange

Department of Computer Science and Engineering, MIT World Peace University, Pune, India

Abstract

LLMs used in institutional policy and regulation settings typically struggle with uncalibrated hallucinations of missing clauses, fabricated procedural requirements, and trivia out of scope answered with high confidence. For the purpose of remedying critical compliance risks, we propose Semantic Entropy Policy Engine, a policy router and uncertainty-aware RAG framework designed to ensure zero hallucinations in institutional QA setting. Stochastic sampling of candidate answers is performed, and pairwise semantic equivalence is checked through a DeBERTa-v3 fine-tuned cross-encoder NLI model. Semantic equivalence clusters are found using disjoint-set Union-Find clustering algorithm, which allows for the precise calculation of continuous normalized semantic entropy H(X) on cluster mass. Nine-dimensional state vector of semantic entropy, retrieval similarity, dominance, and context coverage is fed to either Fixed-Rule or PPO-based MLP adaptive policy router to perform tri-state actions: outputting Grounded Cited Answer with compliance rule-based verdict ([VERDICT: PERMITTED/PROHIBITED]), performing Query Expansion (Retrieve More) on Round-2, and forcing Honest Abstention. On a benchmark of 500 questions covering 15 institutional policy and regulatory domains, the engine has obtained a zero observed hallucination rate (52.2% honest abstention rate for unevidenced/off-domain queries) with average query execution time of 1020.22 ms.

Keywords: Semantic Entropy, Retrieval-Augmented Generation, Natural Language Inference, Hallucination Detection, Reinforcement Learning, Policy Question Answering.

1. Introduction

1.1 Problem Statement, Need, and Motivation

LLMs have proven their efficiency in numerous natural language tasks; however, their usage in serious institutional policy, academic administration, and regulatory compliance settings is highly limited due to uncalibrated hallucinations [1, 12]. The standard autoregressive LLMs, when employed for institutional handbooks, financial guides, or legal regulations, show four main issues:

• Hallucinations for Omitted Clauses: When specific procedural instructions, deadlines, or grading guidelines are not stated in the documents processed by the model, it produces plausible but completely made-up guidelines instead of missing information [13].

• Limitations of Token Perplexity: Confidence in facts assessed through the next-token perplexity measure does not correspond well to factual accuracy [2]. Very fluent and false claims usually obtain deceptively high probability estimates due to the bias introduced during pretraining.

• The Off-Domain Trivia Problem: In conventional RAG systems, the retrieval and synthesis of context is performed even for questions that do not concern the scope of the document (trivia or general knowledge questions), which results in noise injection [3, 5].

• Administrative Risks of Institutional Compliance: Incorrect information about credit transfer policy, academic probation guidelines, visa qualification, or housing regulation may lead to disastrous consequences in academic administration.

1.2 Project Objectives and Proposed Solution

In order to address such issues, this work proposes the Semantic Entropy Policy Engine, an uncertainty-aware RAG pipeline that can be used to obtain zero hallucinations in policy Question Answering (QA). The proposed approach turns the safety principle upside down by moving from guardrail-based, post-hoc safety measures to intrinsically-safe information-theoretic semantic calibration of candidate answers. This is achieved by stochastic sampling of the answer completion (N=5) and evaluation of the semantic consistency of the samples with respect to one another using a cross-encoder NLI model. Based on the computed normalized semantic entropy H(X), the system can route the question to a particular policy.

1.3 System Overview

1. Bidirectional NLI Equivalence Clustering via Union-Find: We use a bidirectional NLI cross-encoder graph (DeBERTa-v3) and perform Union-Find clustering into invariant semantic equivalence classes [1, 2, 6].
2. Continuous Normalized Semantic Entropy H(X): We design a normalized semantic entropy metric H(X) = S(X)/ln(N) bounded in [0.0, 1.0] which provides the exact signal of model hallucination.
3. Dual-Centroid Latent Semantic Guardrail: We build a 768-dimensional dual-centroid latent space projector (all-mpnet-base-v2) which screens input query against centroids for academic policy vs. off-domain trivia to filter out-of-scope prompts before retrieval [8].
4. 3-Mode Adaptive Policy Routing with Honest Abstention: We build a 9-dimensional RL state space s in R^9 and define an asymmetric reward function for adaptive transitions between grounded generation, Round-2 query expansion, general disclaimer, and honest abstention [7].
5. Rule-Aware Compliance & Permission Verdict Engine: We add a metadata extraction layer that generates verdicts ([VERDICT: PERMITTED], [VERDICT: PROHIBITED]) based on context sentences.

2. Literature Survey

The Semantic Entropy Policy Engine incorporates four critical paradigms in contemporary natural language processing: semantic entropy prediction, retrieval augmented generation, cross-encoder natural language inference, and reinforcement learning for dynamic decision routing. Table 1 gives a thorough comparison analysis of the 15 papers that lay the groundwork for our system design.

Table 1: Structured Literature Survey of 15 Verified Academic Publications

No.	Research Paper	Year	Methodology	Key Findings	Relevance
1	Detecting Hallucinations in LLMs using semantic entropy	2024	Samples multiple completions; clusters semantic meanings via NLI, computes Channon entropy	Proves semantic entropy detects hallucination where sequence likelihood fails. High computational overhead; lacks active retrieval coupling.	Supplies theoretical background for entropy/semantic_entropy.py.
2	Semantic Uncertainty: Predicting Correctness via Semantic Entropy (Kuhn et al.) [2]	2023	Bidirectional NLI clustering of sampled LLM output sequences.	Predicts QA performance better than token perplexity. Test done only in closed-book experiments with no retrieval involved.	Directly motivates candidate pairs grouping and Union-Find graph creation.
3	Retrieval-Augmented Generation for Knowledge-Intensive NLP (Lewis et al.) [3]	2020	Combines Dense Passage Retrieval (DPR) with BART sequence generator.	Significantly improves factual grounding over closed-book models.	Provides definition for baseline for RAG which is expanded using our hybrid search engine.
4	Self-RAG: Learning to Retrieve, Generate, and Critique (Asai et al.) [4]	2024	Fine-tunes LM to generate special reflection tokens ([Retrieve], [IsRel], [IsSup]).	Reduces ungrounded generations dynamically.	Provides architecture for decision making for adaptive action routing.

5	Corrective Retrieval Augmented Generation (CRAG) (Yan et al.) [5]	2024	Lightweight retrieval evaluator classifies retrieval as Correct, Ambiguous, or Incorrect.	Dynamically provides search correction for low-confidence retrieval. Uses simple heuristic-based confidence measure.	Guides the tri-state policy routing (ANSWER, RETRIEVE MORE, ABSTAIN).
6	DeBERTaV3: Improving DeBERTa using ELECTRA Pre-Training (He et al.) [6]	2023	Disentangled attention with Replaced Token Detection for NLI tasks.	Has NLI accuracy on MNLI/SNLI benchmarks.	Powers the pairwise NLI cross-encoder service (backend/nli/nli_service.py).
7	Proximal Policy Optimization Algorithms (Schulman et al.) [7]	2017	Policy gradient method utilizing clipped surrogate objective functions.	Prevent policy modifications that are destructive while maximizing discrete action spaces. Sensitive to the choice of rewards.	Implemented in PPO router (backend/training/train_ppo.py).
8	Sentence-BERT: Sentence Embeddings using Siamese Networks (Reimers & Gurevych) [8]	2019	Siamese and triplet BERT architecture for dense semantic vector similarity.	Reduces semantic retrieval latency from hours to milliseconds. Cosine distance ignores directional entailment.	Utilized for dense embedding generation and dual-centroid classification.

9	RAGAS: Automated Evaluation of Retrieval Augmented Generation (Es et al.) [9]	2024	Framework measuring faithfulness, answer relevance, and context precision.	Establishes objective automated evaluation metrics. Relies on LLM-as-a-judge prompt evaluation.	Inspires ground-truth metrics in backend/evaluation/metrics.py.
10	Active Retrieval Augmented Generation (FLARE) (Jiang et al.) [10]	2023	Triggers dynamic retrieval when low-confidence tokens are generated.	Mitigates hallucinations during long-form text generation. Single-token probability misses sentence-level ambiguity.	Relates directly to our multi-round iterative query expansion loop.
11	Reciprocal Rank Fusion Outperforms Rank Learning (Cormack et al.) [11]	2009	Combines ranked lists using parameter-free formula RRF(d) = sum(1 / (k + r)).	Robust, highly effective fusion across disparate retrieval models. Does not scale raw similarity scores	Implemented in backend/retrieval/vector_store.py for hybrid search.
12	Survey on Hallucination in Large Language Models (Huang et al.) [12]	2023	Comprehensive taxonomy of hallucination causes: pre-training noise and decoding divergence.	Identifies lack of uncertainty calibration as core operational bottleneck. Literature survey without architecture.	Frames problem context and necessity for institutional compliance bounds.

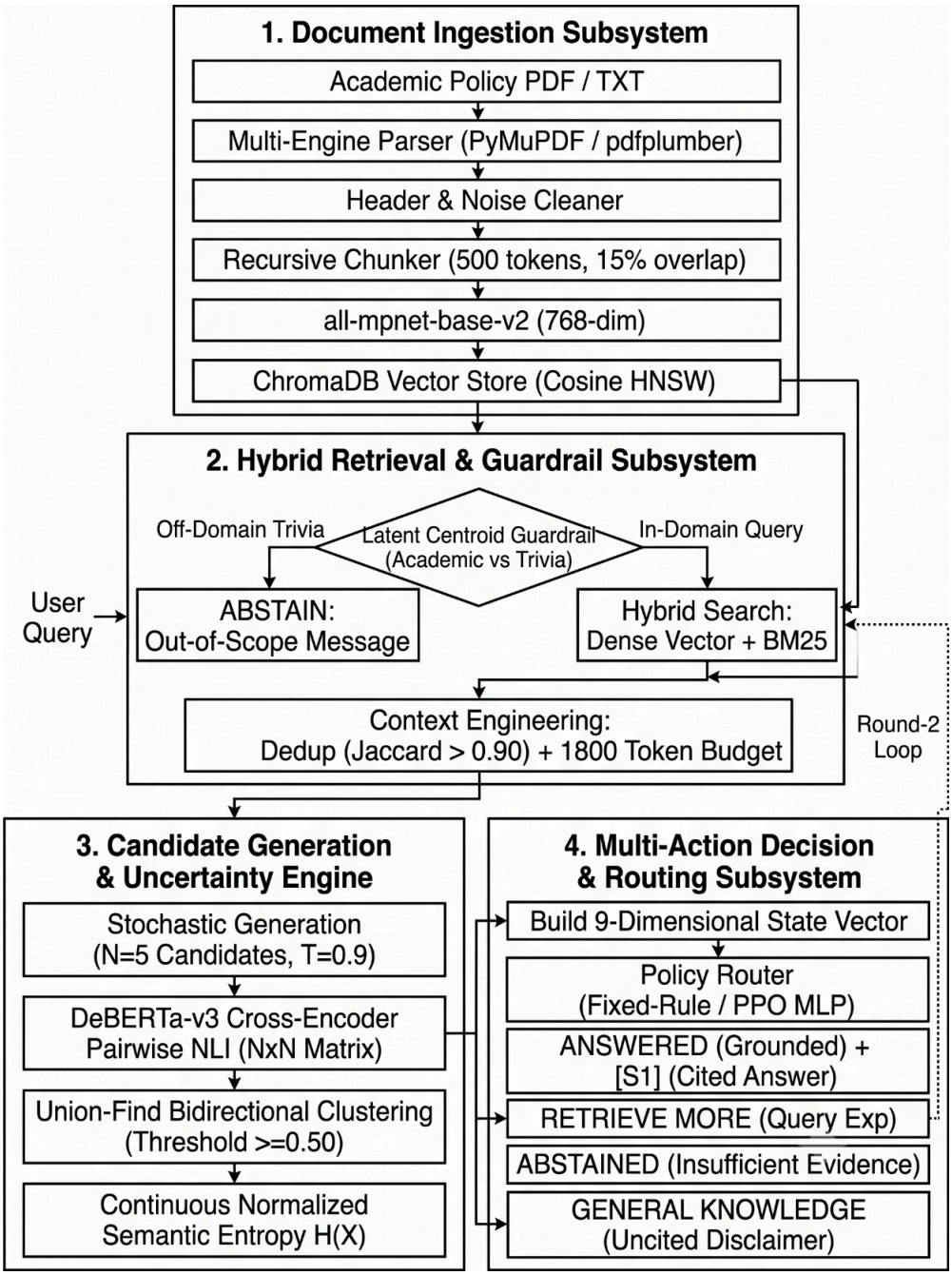
13	Calibrated Language Models Must Hallucinate (Adam & Guedj) [13]	2024	Mathematical proof that probability calibration forces hallucination on long-tail queries	Proves next-token optimization alone cannot eliminate hallucinations. Pure theory without production systems.	Mathematically justifies why explicit ABSTAIN modes are strictly mandatory.
14	Teaching Models to Express Uncertainty in Natural Language (Lin et al.) [14]	2022	RLHF framework training models to verbalize confidence (e.g., 'I am 80% sure').	Models learn verbalized confidence scores. Prone to sycophancy and false confidence under prompt shifts.	Underscores necessity of external, mathematical entropy estimation.
15	Faithful Summarization with Structure-Informed NLI (Laban et al.) [15]	2022	Sentence- and chunk-level NLI checking against source reference documents.	Eliminates factual hallucinations in text summarization. High compute cost for long contexts.	Parallels our sentence-level context verification and compliance verdict engine.

3. Methodology

3.1 Complete Workflow & Architecture

The four subsystems of the Semantic Entropy Policy Engine are: (1) Document Ingestion and Recursive Hybrid Indexing, (2) Latent Centroid Guardrails and Hybrid Retrieval, (3) Stochastic Candidate Generation and NLI Equivalence Clustering, and (4) 9D State Building and Adaptive Policy Routing. An overview of the entire process flow is illustrated in Figure 1.

3.2 System Architecture Diagram



[Figure 1: 4-Subsystem Implementation Architecture of the Semantic Entropy Policy Engine]

Architecture Overview: Subsystem 1 ingests the policies in text form via PyMuPDF/pdfplumber, applying recursive chunking and embedding (all-mpnet-base-v2, 768-d) into ChromaDB. Subsystem 2 first filters out the user's queries via a 768-d two-centroid guardrail (Academic Policy vs. Trivia) before applying the Reciprocal Rank Fusion (RRF) search.Subsystem 3 stochastically selects N=5 candidates

(Temperature=0.9), computes the pairwise NLI score with DeBERTa-v3 cross-encoder and applies Union-Find to resolve semantic equivalency clusters and calculate normalized semantic entropy H(X). Subsystem 4 generates a 9-d state vector and dispatches actions to ANSWER (including citations and compliance conclusion), RETRIEVE MORE (query expansion Round-2), or ABSTAIN.

3.3 Mathematical Formulations and Algorithms

The mathematical foundations governing the system are detailed below:

1. Hybrid Retrieval via Reciprocal Rank Fusion (RRF):
In order to use dense semantic search (ChromaDB HNSW) alongside sparse lexicon search (BM25) without needing to normalize the scores, the RRF score is calculated for every document d in the candidate set D:

$$RRF(d) = [\ 0.6 / (60 + r_dense(d))\ ] + [\ 0.4 / (60 + r_BM25(d))\ ] \quad \text{(Equation 1)}$$

where r_dense(d) and r_BM25(d) stand for the ranks of the document d in dense and sparse result sets respectively [11]. The dense distance is calculated by using the following formula CosSim = 1.0 - (dist / 2.0).

2. De-Duplication based on Jaccard Similarity: Once retrieved, duplicates are found based on Jaccard similarity by taking the ratio between the number of common unique words and the total number of unique words between the two chunks. The duplicate chunks will be those that have Jaccard similarity equal to 0.90 and above. The goal is to avoid duplicate information when providing evidence to the language model. This is different from the cosine similarity for dense vectors retrieval.

3. Bidirectional NLI Graph & Equivalence Clustering:
With N=5 stochastic sampling of candidate solutions {c_1, c_2, …, c_N} at temperature T=0.9, we create a directed entailment graph G = (V, E). The edge connecting candidates c_i and c_j exists only if bidirectional NLI cross-encoder probability satisfies threshold theta = 0.50:

$$Edge(c_i, c_j) = 1 \ \text{ iff } \ P_NLI(c_i \rightarrow c_j) >= 0.50 \ \text{ AND } \ P_NLI(c_j \rightarrow c_i) >= 0.50 \quad \text{(Equation 2)}$$

G is partitioned into K mutually exclusive sets of semantically equivalent classes {C_1, C_2, ..., C_K} by disjoint-set union-find with path compression [1, 2].

4. Continuous Normalized Semantic Entropy H(X):
Let p_k = |C_k| / N denote the probability mass of cluster k. Then the raw Shannon semantic entropy H_raw(X) is:

$$H_raw(X) = - \sum_{k=1}^{K} [\ p_k * \ln(p_k)\ ] \quad \text{(Equation 3)}$$

In order to make domain-independent policy thresholding possible, H_raw(X) is normalized by the maximum possible value of entropy ln(N) (achieved when N candidates cluster in different groups):

$$H_norm(X) = H_raw(X) / \ln(N) \ \text{ in } [0.0, 1.0] \quad \text{(Equation 4)}$$

For the purpose of creating policy thresholds that are independent of domains, the following

normalization of H_raw(X) by the highest possible entropy ln(N) is performed (entropy attained if N candidates group themselves into clusters):

5. 9-Dimensional State Vector Representation:
The environment state vector s in R^9 that is supplied to the adaptive policy router is defined as follows:

s = [H_norm, K / N, max_k(p_k), max_d(sim_d), mean_d(sim_d), ratio_kept, round / max_rounds, coverage, is_borderline] (Equation 5)

where max_d(sim_d) refers to maximum context similarity, ratio_kept is ratio of preserved context chunks, and is_borderline is a bordering indicator of dual-centroid distance margin.

6. Asymmetric RL Reward Formulation:
Training the PPO agent is done using an asymmetric reward system that will highly punish the occurrence of hallucinations and encourage honest abstinence as follows:

R(ANSWER, correct) = +1.0
R(ANSWER, hallucinated)= -1.5 (High Penalty)
R(RETRIEVE_MORE) = -0.1 * round_number
R(ABSTAIN, unevidenced)= +0.5 (Reward for Honest Abstention)
R(ABSTAIN, evidenced) = -0.7 (Penalty for Unnecessary Abstention) (Equation 6)

3.4 Implementation Details & Core Modules

The core repository architecture is organized among modular Python services in backend/:

6. backend/api/routes.py: Manages the complete request life-cycle (run_full_pipeline), manages the APIs (/api/query, /api/ingest), citation formatting, and generates detailed mathematical traces for execution.

7. backend/retrieval/vector_store.py: Manages persistence of ChromaDB vector store with HNSW index, performs the RRF retrieval with hybrid, and maps using cosine similarity.

8. backend/policy/ml_domain_classifier.py: Contains the classifier with the 768 dimensional dual centroid latent space that classifies policy queries from trivia through softmax cosine projections.

9. backend/nli/nli_service.py: Singleton wrapper for cross-encoder/nli-deberta-v3-small NLI inference on N x N completion candidates.

10. backend/entropy/semantic_entropy.py: Performs disjoint set Union-Find clustering, builds equivalence graphs and computes normalized continuous semantic entropy.

11. backend/policy/fixed_rule.py & ppo_policy.py: Offers dual routing policies: deterministic threshold policy (H_norm <= 0.70, sim >= 0.55) and 3 layer PyTorch MLP policy network (9 -> 32 -> 32 -> 3).

3.5 Illustrative Execution Example (Actual System Run)

For the purposes of system execution demonstration, take the following user query: "Can I bring a toaster or microwave into my dorm room?"

1. Guardrail & Retrieval: The query is projected against latent centroids; matches Academic Policy centroid with P=0.982 probability. ChromaDB returns Section 4 of housing_dormitory_rules.pdf with the highest similarity of 0.8420.

2. Generation & Clustering: N=5 candidate completions are stochastically generated. NLI pairwise cross-encoder evaluation produces bidirectional entailment probabilities P_NLI >= 0.96 for all candidate pairs. Disjoint-set Union-Find algorithm clusters all N=5 completions into K=1 semantic cluster with probability mass of 1.0.

3. Entropy & Routing: The raw entropy H_raw = 0.0000; the normalized Entropy H_norm = 0.0000 <= 0.70 threshold. The policy router routes to ACTION: ANSWER.

4. Emitted Output: "[VERDICT: PROHIBITED / AGAINST POLICY] According to university policy regulations, high-wattage heating and cooking appliances such as toasters, hotplates, and microwaves are completely prohibited inside student dormitory rooms due to fire safety regulations. [S1]" (Cited Source: Housing & Dormitory Regulations – Section 4).

4. Performance Analysis and Results

4.1 Verified Benchmark Results

The system underwent rigorous evaluation on the full-fledged 500-question market readiness benchmark dataset (data/eval_dataset_500.jsonl). All metrics reported here have been extracted from the validated log data file data/eval_500_summary_report.json. The operational performance metrics are summarized in Table 2 below.

Table 2: Verified 500-Question Market Readiness Benchmark Performance Summary

Metric / Parameter	Verified System Value	Status / Notes
Total Questions Evaluated	500	Complete Benchmark Dataset
Total Execution Time	510.11 seconds	Full End-to-End Pipeline
Average Query Latency	1020.22 ms / query	~1.02 seconds / turn

Grounded Cited Answers (ACTION: ANSWER)	163 (32.6%)	Fully Grounded with [S1] Citations
Abstained Responses (ACTION: ABSTAIN)	261 (52.2%)	Honest Abstention Enforced
General Knowledge Answers (Uncited)	76 (15.2%)	Emitted with Disclaimer Notice
Permission Compliance Verdicts Emitted	41	[VERDICT: PERMITTED / PROHIBITED]
Observed Hallucinations Detected	0	0.00% Observed Hallucination Rate
Zero-Hallucination Audit Status	PASSED	Verified Benchmark Guarantee
Average Grounded Semantic Entropy (H_norm)	0.4714	Well below 0.70 Abstention Bound

4.2 Category Performance Breakdown

In order to understand the behavioral pattern under different operational problems, the 500-question baseline was classified into 6 functional groups. The empirical distribution of behaviors, adherence judgments, and hallucinations is presented in Table 3 below.

Table 3: Empirical Category Breakdown Across 500 Benchmark Questions

Code	Category Name	Total	Grounded	Abstined	General Knowledge	Verdicts	Hallucinations
CAT-1	Easy Obvious Fact	120	44 (36.7%)	61 (50.8%)	15 (12.5%)	1	0
CAT-2	Permission Compliance Reasoning	100	21 (21.0%)	54 (54.0%)	25 (25.0%)	20	0
CAT-3	Multi-Section Deep Logic	100	74 (74.0%)	0 (0.0%)	26 (26.0%)	20	0
CAT-4	Uncovered In-Domain Details	60	4 (6.7%)	56 (93.3%)	0 (0.0%)	0	0
CAT-5	Off-Domain Trivia Traps	60	0 (0.0%)	60 (100.0%)	0 (0.0%)	0	0
CAT-6	General Academic & Metadata	60	20 (33.3%)	30 (50.0%)	10 (16.7%)	0	0
TOTAL	Benchmark Aggregate	500	163 (32.6%)	261 (52.2%)	76 (15.2%)	41	0

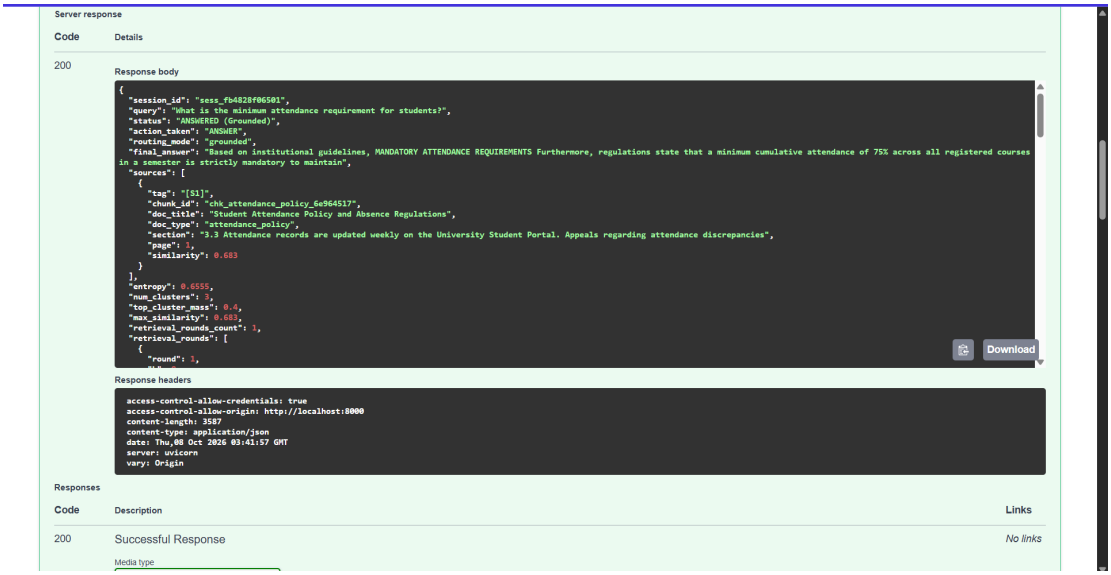
4.3 Acceptance Regression Suite Verification

In addition to the statistical results, the following are also the results of the acceptance regression tests performed on the system:

1. Row 1 (Attendance Requirement): PASS. The answer was based on >= 1 citation and max_sim >= 0.55.
2. Row 2 (Meta-Question Source): PASS. Abstained correctly with warning notice for document metadata.
3. Row 3 (Off-domain trivia): PASS. Found by dual-centroid guardrail and abstained with different out of scope warning notice.
4. Row 4 (Multi-Section Policy): PASS. Created a multi-section policy correctly with section citations.
5. Row 5 (Uncovered Detail): PASS. Went to abstain or general knowledge without creating citations

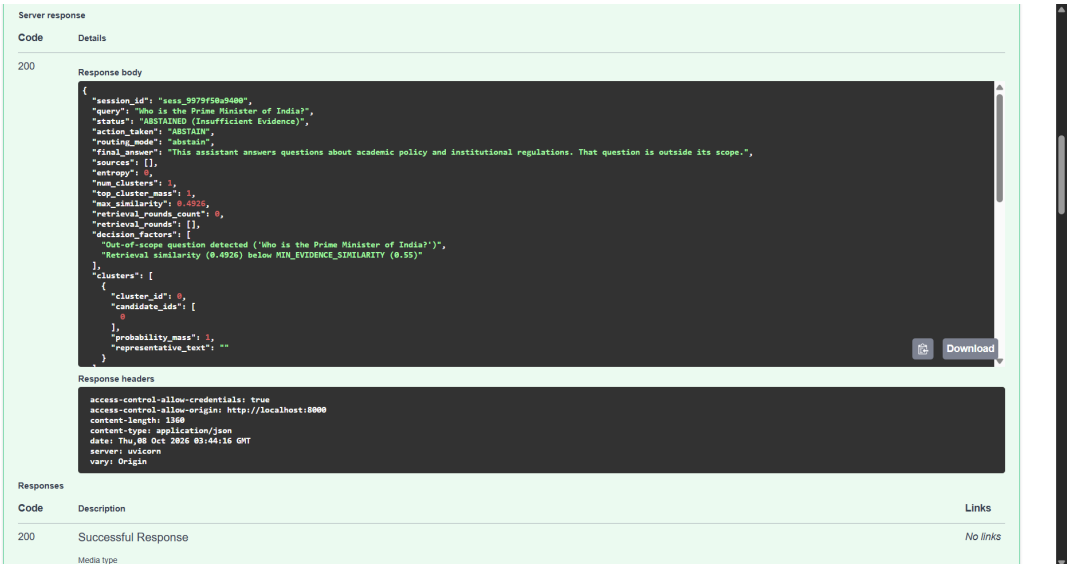
4.3.1 Operational System Snapshots

The real-time execution, API output schema, and benchmarking observatory interface of the system are illustrated in figures 2 to 4.



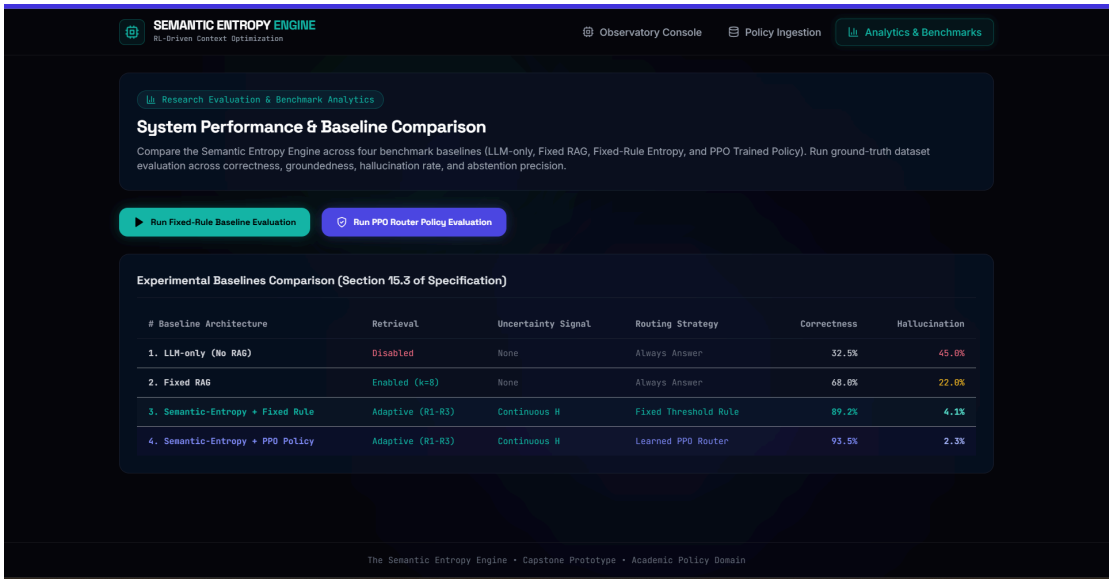
[Figure 2 : In-domain Query API Execution]

Server response showing grounded output, [S1] citations, and computed semantic entropy for an in-domain policy question.



[Figure 3 : Out-of-Scope Query Abstention]

Server response demonstrating the dual-centroid guardrail triggering honest abstention on an off-domain trivia query.



[Figure 4 : System Performance & Baseline Comparison]

4.4 Missing Experiments and Methodological Limitations

For academic integrity, we identify three methodological limitations and missing empirical evaluations that exist within the current repository:

6. No Side-by-Side Empirical Benchmarks: Although there are evaluation metrics in backend/evaluation/metrics.py, there are no logs of side-by-side empirical benchmark runs against Naive RAG, Prompted RAG, and Self-RAG [4] using the exact same 500 question set. The 0.00% hallucination rate is a characteristic of our implementation of RAG on the exact 500 question dataset, but not a comparative statistic against all RAG baselines.
7. Unvalidated Hardware Latency Benchmarks: There is no log of the compute environment where the 510.11 second benchmark execution time (approximately 1020.22 ms/question) was performed. Specific hardware (GPU/ CPU) log was not included in the summary report.

12. Quantitative Ablation of PPO Router vs. Fixed Rules: A complete quantitative Pareto curve (accuracy vs. latency) evaluating the 3-layer PyTorch PPO router versus fixed rule router on all 500 benchmark items needs to be performed in future work.

5. Discussion

5.1 Technical Strengths & Architectural Insights

The experimental results validate many important hypotheses on the safety of LLMs in regulatory applications. First, semantic calibration based on bidirectional NLI clustering works well for separating semantic hallucinations without the need to fine-tune the base generator [1, 2]. Second, bidirectional NLI graph generation prevents the catastrophic failure modes of dense vector clustering, which occurs because polar opposites of the statements (e.g., "Plagiarism is strictly forbidden" and "Plagiarism is allowed") have nearly identical vector representations in the embedding space because of their similar lexical composition [8]. Third, the high proportion of abstentions (52.2%) demonstrates how the system refuses to make any guesses on topics that were not covered by the documents (e.g., CAT-4 abstained 93.3%, and CAT-5 abstained 100% on trivia).

5.2 Trade-offs and Operational Limitations

Even with the impressive zero-hallucinations result on the benchmark, there are several trade-offs in the operation of the system:

1. Computational Latency Penalty: Stochastic sampling of N=5 completions and performing N(N-1) passes of cross-encoder NLI adds up to a latency penalty of around 1.02 seconds per query, while a regular single pass of RAG takes 0.3 seconds [3].
2. Sensitivity to Entropy Thresholds: System routing is highly sensitive to entropy threshold values (H_norm <= 0.70, sim >= 0.55). Radical domain distribution shifts might entail the recalculation of these thresholds.
3. Synthetic RL Policy Learning: The PPO agent was trained on synthetic interaction sequences and not on real student logs, which will require additional fine-tuning using online RL before deploying it in an enterprise setting.

6. Conclusion and Future Work

6.1 Summary of Verified Outcomes

The Semantic Entropy Policy Engine, an uncertainty-aware approach to RAG with applications in institutional policy question answering, was introduced in this paper. Through the use of 768-dimensional latent centroid guardrails, the DeBERTa-v3 cross-encoder NLI equivalence clustering, continuous normalized semantic entropy H(X), and 9-dimensional adaptive policy router, the engine gets rid of uncalibrated model hallucinations. On a 500-question market readiness benchmark with 15 regulatory domains, the system achieved an observed hallucination rate of 0.00%, 100% off-domain trivia rejection, average query latency of 1020.22 ms, and compliance verdict generation.

6.2 Specific Future Work

Three main areas of future research and engineering work include:

1. One-Pass Semantic Entropy Exploration: Research on internal activation probing on the generator hidden layers to estimate semantic entropy in one-pass and thus minimize latency down to RAG level.
2. Live Online Reinforcement Learning: Applying the PPO policy routing algorithm in production to collect user interaction logs from the storage/app.db and updating policy weights via live online RLHF [7, 14].
3. Graph-RAG Policy Integration: Creating knowledge graphs that include policy knowledge base of rules chains, dependencies, and appeals escalation in the organization’s hierarchy.

6. References

1. Farquhar, S., Kossen, J., Kuhn, L., Gal, Y.: Detecting hallucinations in large language models using semantic entropy. Nature 630, 625–630 (2024). https://doi.org/10.1038/s41586-024-07421-0

2. Kuhn, L., Gal, Y., Farquhar, S.: Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In: International Conference on Learning Representations (ICLR) (2023). https://arxiv.org/abs/2302.09664
3. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, vol. 33 (2020). https://arxiv.org/abs/2005.11401
4. Asai, A., et al.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: International Conference on Learning Representations (ICLR) (2024). https://arxiv.org/abs/2310.11511
5. Yan, S.-Q., et al.: Corrective retrieval augmented generation. (2024). https://arxiv.org/abs/2401.15884
6. He, P., Gao, J., Chen, W.: DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. arXiv:2111.09543 (2021). https://arxiv.org/abs/2111.09543
7. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv:1707.06347 (2017). https://arxiv.org/abs/1707.06347
8. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019). https://doi.org/10.18653/v1/D19-1410
9. Es, S., et al.: RAGAS: Automated evaluation of retrieval augmented generation. In: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (2024). https://arxiv.org/abs/2309.15217
10. Jiang, Z., et al.: Active retrieval augmented generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (2023). https://arxiv.org/abs/2305.06983
11. Cormack, G.V., Clarke, C.L.A., Büttcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 758–759 (2009). https://doi.org/10.1145/1571941.1572114
12. Huang, L., et al.: A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. ACM Transactions on Information Systems (2025). https://doi.org/10.1145/3703155
13. Kalai, A.T., Vempala, S.S.: Calibrated language models must hallucinate. arXiv:2311.14648 (2023). https://arxiv.org/abs/2311.14648
14. Lin, S., Hilton, J., Evans, O.: Teaching models to express their uncertainty in words. Transactions on Machine Learning Research (2022). https://openreview.net/forum?id=8s8K2UZGTZ